

\$ cat notes.md

DevOps & Cloud [SWE] — Class 9 Notes

Lecture Title	Kubernetes Fundamentals
Date/Time	01/09/2026, 11:00 AM
Duration	120 minutes
Instructor	Ritesh Prajapati

WHAT'S INSIDE

📅 Class Logistics & Recap of Docker Networking/Volumes

🔧 Dockerfile Instructions — Rapid-Fire Recap Quiz

📋 Docker Compose — What It Is & Why

📄 YAML Basics

📄 Anatomy of a `docker-compose.yaml` File

🖥️ Live Demo — Building the 3-Tier Compose App

📋 Is Docker Compose Actually Used in Industry? — A Class Debate

🔄 Docker Compose — Command Recap

🔗 Full-Course Docker Recap (Transition to Kubernetes)

✳️ Kubernetes — Why It Exists

✳️ Kubernetes Architecture — Control Plane & Worker Nodes

✳️ Kubernetes Control Plane — The Four Core Components

✳️ Kubernetes Worker Node — Kubelet, Kube-Proxy & Container Runtime

🏆 Certifications & Career Aside

🖥️ Hands-On Setup for Next Time — Minikube

📖 Appendix — Full Command & Concept Reference

📄 Cross-Reference — This Class's Working File

📄 A Note on Sources & Transcript Quality

Structured from the raw class transcript — nothing cut, just organized.

01 📅 Class Logistics & Recap of Docker Networking/Volumes

- **Recap quiz on [Class-08](#)** (networking & volumes), rapid-fire:
- None network → has no driver at all (that's exactly why it's called "none"); cannot reach Google or any external server.
- Host network → shares your local machine's network directly with the container, confirmed via `docker run -dit --network host --name <name> nginx` — no `-p` needed, since there's no interface between the container and host to bridge.
- Two volume types: the default **Docker volume**, and **bind mount** (used specifically when you want to bind your current/local folder into a container).
- Volume flag recap: `-v <volume-name>:<path>` when running a container; `docker volume create <name>` to create one; `docker network create <name>` for a network.
- Confirmed again: containers in the *same* network can talk to each other; containers split across two different **user-defined** networks cannot, by design (this was the point of the "create 3 networks" homework from Class-08).
- **New submission process explained:** a shared task/documentation set has been added to the "DevOps Heroes" repo, organized **per session**, with a form (linked per section — A or B) where each student submits their `README.md` link for that session's homework. Each session folder should contain one `README.md` with a short task description + a screenshot per task.

• • •

02 🎮 Dockerfile Instructions — Rapid-Fire Recap Quiz

A quick check on Dockerfile fundamentals before introducing Compose, since the two are easy to confuse:

- **FROM** → base image. **RUN** → runs a command at build time. **CMD** → the triggering/default command run at the end of the Dockerfile.
- **EXPOSE** — re-confirmed as *not* the same as actually publishing a port. `EXPOSE 80` just tells you the app is listening on port 80 internally; it doesn't put anything on port 80 in your browser. Framed explicitly around why a DevOps engineer cares: when building a CI/CD pipeline for a developer's app, `EXPOSE` is how you find out which port that component is actually listening on, so you know what to redirect traffic to.
- **COPY** — local-only, copies files from your build context. **ADD** — copy plus the extra ability to pull in an external resource directly (a zip/RPM file from a link).
- **ENTRYPOINT** vs. **CMD** — recap: `ENTRYPOINT` **cannot** be overridden; `CMD` **can**.

- **Multi-stage Dockerfile, recap + an explicit clarification:** builder stage (`AS builder` on the first `FROM` line) → runtime stage (`--from=builder` to copy specific output, e.g. a `dist` folder with its `index.js`). **Explicitly called out: a multi-stage Dockerfile still only ever produces ONE container.** This is deliberately contrasted with today's new topic — don't confuse "multi-stage" (one Dockerfile, one final container) with "multi-container" (Docker Compose, many containers at once).

• • •

03 🎵 Docker Compose — What It Is & Why

- **The motivating problem, restated concretely:** deploying even a simple 3-container app (frontend, backend, database) manually requires: creating a network, attaching the frontend, doing the same for backend, doing the same for the database, then connecting the database container to *both* the backend's network and — wait, ensuring frontend can't reach the database directly. That's a lot of repeated manual steps for one small app.
- **Docker Compose = a YAML file describing all of it at once.** Once written, a single command (`docker compose up`) builds and starts every container it describes — 3, 4, 5, however many. You can manage all of them together: bring them up, take them down, scale them.
- Historical framing: before Kubernetes existed, **Docker Swarm** filled a similar "manage many containers" role, and Compose-style workflows were more central to production use back then.

• • •

04 📄 YAML Basics

- Described as roughly as easy as Python, if you already know Python: **indentation-based**, and conceptually just **key-value pairs**, **arrays**, and **dictionaries** — nothing more exotic than that.
- File naming convention: `docker-compose.yaml` (or `.yaml`) is today's standard; older/legacy setups sometimes used `compose.yaml` , but modern (2024+) convention favors the `docker-compose.yaml` name.

• • •

05 Anatomy of a `docker-compose.yml` File

Top-level structure: `services`, `networks`, `volumes`.

- Each entry under `services:` is called a **service** (not "container name," even though functionally it plays that role — equivalent to the `--name` you'd give with `docker run`).
- **Per-service options:**
 - `image: <name>` — pull a ready-made image (e.g. `nginx`, `mysql:8.0`) from a registry, same as `docker pull`.
 - `build: ./<folder>` — instead of pulling an image, build one from your own Dockerfile located in that folder (Compose automatically looks for a `Dockerfile` there).
 - `ports: ( host:container )` — only for services that genuinely need external access. **Explicit rule stated repeatedly: frontend gets a `ports:` entry; backend and database (almost) never do.** You don't expose a backend or database directly to the internet — at most, in real systems, only a small, deliberately chosen set of API routes get exposed, and even that's flagged as generally bad practice outside of testing.
 - `environment:` — environment variables (e.g. `MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE`) — same idea as a Dockerfile's `ENV`, just declared per-service here instead.
 - `volumes:` — either a bind mount or a named Docker volume, written the same way as the `-v` flag (`volume-name:/path` or `./local-folder:/path`).
 - `networks:` — which custom network(s) this service belongs to.
 - `networks:` and `volumes:` also appear as their own top-level keys, outside `services:` — that's where you actually declare the names of any custom networks/volumes referenced inside individual services.
 - `depends_on:` — makes one service wait until another is confirmed up before starting — e.g. `backend` depending on `database`, so Compose won't even attempt to start the backend container until the database container is running.

. . .

06 Live Demo — Building the 3-Tier Compose App

Simple version first — all three services using off-the-shelf images (two Nginx + one MySQL), attached to networks, no custom code yet.

Core Compose commands, demoed live: - `docker compose up` — builds and starts everything, showing all containers' logs together in the foreground. - `docker compose up -d` — same, but detached (background), same idea as `docker run -d`. - `docker compose ps` — lists just the containers this Compose file created (the Compose-scoped equivalent of `docker ps`). - `docker compose down` — stops **and removes** everything the Compose file created. - `docker compose up`

`-d --build` — forces a rebuild of any Dockerfile-based (`build:`) services before starting — needed whenever you've changed code/Dockerfile content for a custom-built service. - **Gotcha discovered live:** Compose commands only work when run **from inside the folder containing the `docker-compose.yaml` file**. Plain `docker` commands (`docker ps` , `docker images` , etc.) work from any directory; `docker compose` commands do not.

Full-featured version, actually run and inspected in class (frontend + backend + database, all wired together deliberately): - **frontend** — `nginx:latest` image, `ports: 8080:80` , **bind-mounted** custom `index.html` and a custom `nginx.conf` directly over Nginx's defaults (same bind-mount trick as Class-08 §12 — replacing Nginx's default welcome page with a real page), attached to a `frontend` network. - **backend** — **built from its own Dockerfile** (`build: ./backend`) rather than pulled — a minimal Python app (`app.py`), `EXPOSE 5000` (again: just documentation, not exposure), attached to **both** the `frontend` network *and* the `backend` (database) network, since backend is the only service that legitimately needs to talk to both sides. `depends_on: database` — Compose waits for the database container before starting this one. - **database** — `mysql:8.0` image, `environment:` block for root password + DB name, a **named volume** mounted at MySQL's default data path (`/var/lib/mysql`) so data survives container recreation (same mechanism as Class-08 §11), attached only to the `backend` network. - **Verified live, exactly matching the "never expose backend" rule:** - `docker exec -it <frontend-container> bin/bash` , then `curl http://backend:5000/api` — worked, returning a JSON response — proving frontend *can* reach backend **internally**, by service name (the same DNS-via-user-defined-network resolution from Class-08). - Directly hitting `localhost:5000` from the *host* browser — did **not** work, since the backend's port was never mapped to the host. This contrast was the actual point of the demo: internal service-to-service access works, external public access to backend does not, by design. - Mentioned in passing: this is effectively an **Nginx reverse proxy** setup — Nginx's `proxy_pass` config redirects `/api` -style requests from the frontend container to the backend service by name.

. . .

07 🎵 Is Docker Compose Actually Used in Industry? — A

Class Debate

A genuinely substantive back-and-forth worth preserving, not just banter:

- **Instructor's position:** Docker Compose isn't how real production systems run at scale — it's a *learning stepping stone* toward Kubernetes. Most companies at real scale use Kubernetes (or a managed flavor like EKS). Still worth learning thoroughly because Docker/Compose fundamentals are heavily tested in interviews — the instructor mentioned personally having given 100–150+ interviews over their career, with "write a Dockerfile," "write a docker-compose file," and "explain this Compose setup" recurring as genuine, common questions.

- **A student pushed back**, arguing that smaller companies/startups (citing their own company) *do* use Docker/Compose-level tooling in production rather than paying for full Kubernetes at small scale.
- **Instructor's response, a fair concession with a firm line held:** whether to use Kubernetes is genuinely a **scale decision** — running a Kubernetes cluster for a handful of users is overkill ("we call that a blunder"). But the nuance goes further than just "small vs. big": even at **10,000, 100,000, or a million users**, the instructor was explicit that whether Kubernetes is the right call still **"depends very much on the team size, on the course the company is ready to accept"** — it's a continuous judgment call tied to team maturity and company readiness, not a hard threshold you cross at some fixed user count. Shared their own progression as an example: started with plain EC2, then ECS, then ECR, and only rarely reached for EKS specifically because of its cost at small scale. **But regardless of orchestration choice** — Compose, ECS, or Kubernetes — the backend/database-never-exposed-publicly principle holds: real infrastructure separates a **public subnet** (frontend-facing) from a **private subnet** (backend/DB), and backend ports are never opened to the public internet from the private subnet, no matter which tool is orchestrating the containers.

• • •

08 Docker Compose — Command Recap

- `docker compose up` / `docker compose up -d` / `docker compose up -d --build`
- `docker compose ps`
- `docker compose down`
- Must be run from the directory containing the `docker-compose.yaml` file.

• • •

09 Full-Course Docker Recap (Transition to Kubernetes)

Read aloud as a checkpoint before moving on: Docker architecture, container lifecycle, basic commands, the `system prune` family, images/containers/volumes/networks, the two volume types and four network types, container-to-container communication and DNS resolution via user-defined bridge networks, isolating services from each other via separate networks, and finally the multi-stage-vs-multi-container (Compose) distinction — when and why to reach for each. **Direct framing for what's next:** *"If you're able to understand Docker and troubleshoot all the container-specific things, Kubernetes will be much easier — at its base, Kubernetes is just a manager for containers."*

• • •

10 ✳ Kubernetes — Why It Exists

- **Docker Swarm** is named as Kubernetes' direct predecessor — Docker's own built-in clustering tool, used before Kubernetes existed. Real pros/cons discussed:
- Pros: much simpler, cheaper, and easier to learn/manage than Kubernetes.
- Cons: real problems with **scaling**, **rollback**, and handling **complex/legacy deployments** — good enough for small applications, not for large or complex ones.
- **Why Kubernetes specifically:** built by Google engineers, later open-sourced, designed from the start to scale to millions/billions of nodes — deploying, scaling, managing, *and* monitoring microservices and legacy applications alike, not just running them.
- **The natural progression, stated explicitly:** one container → Docker; several containers on one host → Docker Compose; tens of containers → Docker Swarm; massive, distributed, production scale → **Kubernetes**.
- **"K8s"** — the name comes from "Kubernetes" having 8 letters between the K and the s (a repeated "funny fact" callback from earlier classes).
- **Managed Kubernetes per cloud provider:** **EKS** (AWS), **AKS** (Azure), **GKE** (Google Cloud).

• • •

11 ✳ Kubernetes Architecture — Control Plane & Worker

Nodes

- Same conceptual shape as Docker's client-server split, but Kubernetes' version has **two main components**: the **control plane** (master — "the boss"/"the brain") and the **worker node(s)** ("the servant" — where your applications actually run).
- **Critical distinction, called out directly:** the control plane runs **no application workloads of its own** — its entire job is managing and scheduling everything else.
- **Flagged explicitly as a common interview question:** if the control plane crashes, your already-running applications may keep running for a while, but the cluster loses its ability to manage/schedule/heal anything until it's back.
- **High availability via multiple master nodes** ("quorum") — a real example cited: a production cluster the instructor works with runs **3 master nodes** managing roughly **7,200 services**, so that if one master fails, the others take over its workload.
- **The core terminology ladder** (another favorite interview question): **Cluster** → **Node** → **Pod** → **Container**.
- A **Pod** is the **smallest deployable unit** in Kubernetes — it's an environment that can hold one or more containers. In real-world practice, most pods run just **one main container**, sometimes plus a second helper container — described in class as an *"init container, meaning a sidecar."* >

Simplified, since this blurs two distinct things: an **init container** runs *before* your main container starts, does some setup work (e.g. waiting for a dependency to be ready), and then exits for good — it doesn't run alongside the app. A **sidecar container** runs *for the pod's entire lifetime*, alongside the main container (e.g. continuously shipping logs). Both are "helper containers in the same pod" in spirit, which is likely why they got merged together here — but if you see either term used precisely elsewhere (docs, interviews), they're not interchangeable.

- Kubernetes has its own parallel concepts to Docker's volumes and networks: **Kubernetes Volumes** and **Network Policies** — with **service mesh** name-dropped as a genuinely advanced, high-value networking topic for later.

• • •

12 ✳ Kubernetes Control Plane — The Four Core Components

Component	Role
etcd	A key-value store database — holds the entire state of the cluster: every pod, every container's detail, everything. Described directly as " <i>the storage of the complete cluster.</i> "
kube-apiserver	The cluster's front door — literally the only way in. Every request to create, read, or manage anything in the cluster has to go through the API server first; nothing else can be reached directly.
kube-scheduler	Decides which worker node a new pod should run on, based on that node's available resources (CPU, memory) versus what the pod needs. Its job stops at scheduling — it doesn't create anything itself.
controller manager	Continuously watches and reconciles cluster state via different controller types — e.g. a node controller (checks whether expected nodes are actually up), a replica-set controller (ensures the right number of pod replicas exist on a given node, scaling up or down to match).

- (A "cloud controller manager" box was shown on the reference diagram but explicitly waved off as out of scope for now — "forget it, never existed" for today's purposes.)
- **Full request flow, framed for interview purposes** (`kubectl apply -f deployment.yaml` as the example), as described in class: the call hits the **API server** first → the API server asks **etcd** for the resource requirements → the **scheduler** picks a suitable worker node based on available resources → the **controller manager** and the node's own **kubelet** (see §13) work together to actually bring the pod up and keep it matching the desired replica count.

Simplified, since "the API server asks etcd for requirements" is a bit of a shortcut: the resource requirements (how much CPU/memory a pod needs) actually come from your own deployment YAML file, not from etcd. What etcd actually stores is the cluster's current state

— what's already running, where. So the more precise version is: API server receives your request → **writes the desired state to etcd** → the scheduler reads both your pod's stated requirements and etcd's record of each node's current usage, to decide where the new pod fits. Etcd is the cluster's memory, not the source of a pod's own resource requirements.

• • •

13 🏆 Kubernetes Worker Node — Kubelet, Kube-Proxy & Container Runtime

- **kubelet** — the main agent running on every worker node. Its job: keep the actual running container count matching what's expected (e.g. "3 frontend replicas" — if one dies, kubelet recreates it; if there's an extra one, it removes it) and continuously report container health back to the API server as a **heartbeat**.
- **kube-proxy** — handles **networking and network-policy enforcement** for the pods on that node (the node-level counterpart to Docker's networking layer).
- **Pod, redefined precisely:** "a pod is nothing but an environment handled by kubelet and kube-proxy — inside a pod, you run a container."
- **CRI (Container Runtime Interface)** — the actual component that runs containers inside pods. Kubernetes originally used Docker directly for this; today the more common default is **containerd** — described as effectively "the same idea as Docker, just with some custom changes," built specifically to serve as Kubernetes' runtime after Docker's own daemon stopped being a great fit.

• • •

14 🏆 Certifications & Career Aside

- Real Kubernetes certification names mentioned: **CKA, CKAD, CKS, KCNA**, and a fifth (completing all of them earns the informal "**Kubestronaut**" title).
- Cost reality check: each certification runs roughly **\$300–500**, with **two attempts allowed within a year**; a friend who passed all of them now runs a community offering ~80% off through that network.
- **Broader career philosophy shared:** grab free or steeply-discounted certifications the moment they're available — cited a personal example of clearing 6–7 Oracle Cloud certifications (normally \$700+ combined) for free during a limited promotional window, before they reverted to full price a month later. "If you're thinking twice, the chance will pass."

• • •

15 🖥️ Hands-On Setup for Next Time — Minikube

- **Minikube** introduced as the tool for practicing Kubernetes locally, without needing a real multi-server cluster — the natural next hands-on step now that the architecture concepts are covered.
- Core commands: `minikube start` , `minikube stop` , `minikube status` .
- **Recommended primary learning resource: the official Kubernetes documentation** — preferred explicitly over GeeksforGeeks/W3Schools-style sites, since it's maintained directly by the Kubernetes project itself and stays current with the latest version.
- **Homework:** 1. Install Minikube (installation link shared via the session's README). 2. Run `minikube start` , then confirm with `minikube status` . 3. Read through the official documentation pages for each control-plane/worker-node component covered today (API server, etcd, scheduler, controller manager, kubelet, kube-proxy, container runtime).

. . .

16 📖 Appendix — Full Command & Concept Reference

Command / Concept	Purpose
<code>docker compose up</code>	Build and start all services in a Compose file (foreground)
<code>docker compose up -d</code>	Same, detached/background
<code>docker compose up -d --build</code>	Rebuild any <code>build:</code> -based service images before starting
<code>docker compose ps</code>	List containers created by the current Compose file
<code>docker compose down</code>	Stop and remove everything the Compose file created
<code>docker-compose.yaml</code>	The standard Compose file name
<code>services:</code>	Top-level Compose key — one entry per container/service
<code>image:</code>	Pull a ready-made image for a service
<code>build: ./<folder></code>	Build a service's image from a Dockerfile in that folder
<code>ports:</code>	Host:container port mapping — used for externally-reachable services (e.g. frontend) only
<code>environment:</code>	Per-service environment variables
<code>volumes:</code> (service-level)	Mount a bind mount or named volume into a service

Command / Concept	Purpose
<code>networks:</code> (service-level)	Which custom network(s) a service belongs to
<code>depends_on:</code>	Delay starting one service until another is confirmed up
Docker Swarm	Docker's own pre-Kubernetes clustering tool — simpler, but weaker at scaling/rollback
Kubernetes (K8s)	Container orchestrator built for massive-scale deployment, scaling, and management
EKS / AKS / GKE	Managed Kubernetes on AWS / Azure / Google Cloud
Cluster → Node → Pod → Container	The core Kubernetes terminology ladder
Control plane (master)	Manages/schedules the cluster; runs no application workloads itself
Worker node	Where pods/containers actually run
<code>etcd</code>	Key-value database holding full cluster state
<code>kube-apiserver</code>	The cluster's single front door for all requests
<code>kube-scheduler</code>	Decides which node a new pod runs on
Controller manager	Reconciles cluster state (node controller, replica-set controller, etc.)
<code>kubelet</code>	Node agent — keeps actual pod/container counts matching desired state, reports heartbeats
<code>kube-proxy</code>	Node-level networking/network-policy enforcement
CRI / containerd	The container runtime Kubernetes actually uses to run containers inside pods
<code>minikube start</code> / <code>stop</code> / <code>status</code>	Run and check a local single-node Kubernetes cluster for practice

• • •

17 Cross-Reference — This Class's Working File

`Class-09/compose.yaml` (this repo's own file) matches the **simple, first-pass** version of the demo from §6 — three services (`frontend` , `backend` , `database`), all using stock images (two `nginx` , one `mysql:8.0`), with a named volume (`db-data`) for the database's `/var/lib/mysql` path. It does **not** yet include the refinements from the full-featured demo actually walked through later in class: no `networks:` block (so all three services currently share Docker Compose's own default

network rather than being deliberately isolated), no `build:` /bind-mount for a custom frontend page, and no `depends_on`. Worth revisiting against §5–§6 above to bring it in line with the "frontend can't reach database directly" requirement that was the whole point of the lesson.

• • •



A Note on Sources & Transcript Quality

This transcript is almost entirely clean English, with only brief noise at the very start. One thing worth flagging: this session's transcript was generated under a different student's account than the earlier ones in this repo (visible in the raw file's metadata, not reproduced here) — meaning it's very likely from a shared/synced recording rather than a personally-recorded session, though the content itself reads as the same class and instructor throughout, with no gaps apparent in the flow.